

## TP MongoDB

Cours + Pratique

*De zéro à un projet fil rouge CityFlow, avec mongo-express comme interface*

|               |                                                  |
|---------------|--------------------------------------------------|
| Module        | NoSQL — B3 INFO Tronc Commun                     |
| Séance        | Séance 2 (Jour 1 après-midi)                     |
| Durée estimée | 4 à 5 heures (cours + TP)                        |
| Outils requis | Docker Desktop, navigateur web, éditeur de texte |
| Public        | Étudiants B3 INFO (INFRES + CYBSEC)              |
| Pré-requis    | Notions Docker, SQL de base                      |
| Formateur     | KAKE Abdoulaye                                   |
| Année         | 2025 / 2026                                      |

**Comment utiliser ce document ? Ce TP combine du cours et de la pratique en alternance. Les passages « COURS » présentent les concepts. Les passages « VOTRE TOUR » sont des exercices à faire individuellement ou en binôme. Les passages « CityFlow » alimentent le projet fil rouge. Avancez à votre rythme, ne sautez aucune étape.**

## Une brève histoire

MongoDB a été créé en 2009 par **Eliot Horowitz, Dwight Merriman et Kevin Ryan**, fondateurs de la société **10gen** (renommée MongoDB Inc. en 2013). Le nom « Mongo » vient de **humongous** (gigantesque en anglais), reflétant l'ambition initiale de gérer des volumes massifs de données. À l'origine, l'équipe développait une plateforme PaaS et a créé sa propre base parce qu'aucune solution existante ne leur convenait. Ils ont open-sourcé MongoDB en 2009 et l'ont rapidement vu adopté par toute l'industrie tech.

*Aujourd'hui MongoDB est la **base NoSQL la plus utilisée au monde** (selon DB-Engines Ranking). Elle est employée par Adobe, eBay, Forbes, LinkedIn, Cisco, et des dizaines de milliers de startups. C'est pour cette raison que ce cours lui consacre une séance approfondie : c'est la base que vous rencontrerez le plus en stage et en début de carrière.*

## Positionnement technique

|                           |                                                                                             |
|---------------------------|---------------------------------------------------------------------------------------------|
| <b>Modèle</b>             | Document (JSON/BSON)                                                                        |
| <b>Position CAP</b>       | CP (cohérence prioritaire), avec tunable consistency via les write concerns                 |
| <b>Transactions</b>       | ACID sur un document depuis le départ.<br>ACID multi-documents depuis la version 4.0 (2018) |
| <b>Scalabilité</b>        | Horizontale via sharding, réplication via replica sets                                      |
| <b>Langage de requête</b> | MQL (MongoDB Query Language), syntaxe JSON-like                                             |
| <b>Licence</b>            | SSPL (Server Side Public License) depuis 2018 — open-source mais avec restrictions cloud    |
| <b>Version actuelle</b>   | MongoDB 7.0 (utilisée dans ce cours),<br>MongoDB 8.0 disponible                             |

## Partie 1 — Mise en route avec Docker et mongo-express

### 1.1. Objectif de cette partie

À la fin de cette partie, vous aurez **MongoDB qui tourne dans Docker** et **mongo-express** accessible depuis votre navigateur, sans jamais avoir tapé une ligne de mongosh. C'est l'environnement de travail que vous garderez pour toutes les séances NoSQL.

### 1.2. Pourquoi Docker et mongo-express ?

**Docker** vous permet de lancer MongoDB sans l'installer sur votre machine. Tout est isolé dans un conteneur, et vous pouvez tout supprimer en une commande sans laisser de trace. C'est la pratique standard en entreprise pour les environnements de développement.

**mongo-express** est une interface web open-source pour MongoDB. Plutôt que d'apprendre la syntaxe mongosh dès le premier jour, vous allez **cliquer et explorer**. Une fois les concepts acquis, ceux qui veulent peuvent passer à la ligne de commande, mais ce n'est pas nécessaire pour ce module.

**Note importante** : dans un vrai projet, mongo-express est utilisé en **développement uniquement**. En production, on accède à MongoDB via le code applicatif (Python, Node.js, .NET, etc.). mongo-express vous donne ici un raccourci pédagogique pour vous concentrer sur les concepts NoSQL.

### 1.3. Étape 1 : créer le dossier de travail

Créez sur votre machine un dossier dédié au module NoSQL. Toutes les séances utiliseront ce même dossier — on l'enrichira progressivement.

```
# Sur Linux / macOS / Git Bash sous Windows :
mkdir -p ./cityflow-nosql
cd ./cityflow-nosql

# Sur Windows (PowerShell) :
mkdir .\cityflow-nosql
cd .\cityflow-nosql
```

### 1.4. Étape 2 : créer le fichier docker-compose.yml

Dans ce dossier, créez un fichier nommé exactement docker-compose.yml avec votre éditeur préféré (VSCode recommandé). Copiez-y le contenu ci-dessous :

```
services:
  mongodb:
    image: mongo:7.0
    container_name: cityflow-mongo
    restart: unless-stopped
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: cityflow2025
    networks:
      - cityflow-net

  mongo-express:
    image: mongo-express:1.0.2
    container_name: cityflow-mongo-express
    restart: unless-stopped
    ports:
      - "8081:8081"
```

```
environment:
  ME_CONFIG_MONGODB_ADMINUSERNAME: admin
  ME_CONFIG_MONGODB_ADMINPASSWORD: cityflow2025
  ME_CONFIG_MONGODB_URL: mongodb://admin:cityflow2025@mongodb:27017/
  ME_CONFIG_BASICAUTH_USERNAME: student
  ME_CONFIG_BASICAUTH_PASSWORD: nosql2025
depends_on:
  - mongodb
networks:
  - cityflow-net

volumes:
  mongo_data:

networks:
  cityflow-net:
    driver: bridge
```

### Comprendre ce fichier ligne par ligne

- ``services:`` — déclare les deux conteneurs qu'on va lancer ensemble
- ``mongodb`` — le service MongoDB lui-même, image officielle version 7.0
- ``ports: "27017:27017"`` — expose MongoDB sur le port 27017 de votre machine
- ``volumes: mongo_data:/data/db`` — persistance : les données survivent à un redémarrage du conteneur
- ``MONGO_INITDB_ROOT_USERNAME/PASSWORD`` — identifiants admin créés au premier lancement
- ``mongo-express`` — l'interface web qui se connecte automatiquement à MongoDB
- ``ME_CONFIG_BASICAUTH_*`` — protection par mot de passe pour accéder à mongo-express
- ``depends_on: mongodb`` — mongo-express attend que MongoDB soit lancé avant de démarrer
- ``networks: cityflow-net`` — réseau Docker privé où les deux conteneurs se voient par leur nom

### 1.5. Étape 3 : lancer la stack

```
# Dans le dossier cityflow-nosql, lancer en arrière-plan
docker compose up -d

# Vérifier que les deux conteneurs sont "Up"
docker compose ps
```

Vous devez voir deux conteneurs à l'état **Up** : cityflow-mongo et cityflow-mongo-express. Si l'un est Restarting ou Exited, regardez les logs avec :

```
docker compose logs mongodb
docker compose logs mongo-express
```

Container CPU usage ⓘ

0.71% / 1400% (14 CPUs available)

Container memory usage ⓘ

118.22MB / 7.47GB

Show charts

Q Search

☰ Only show running containers

| <input type="checkbox"/> |   | Name           | Container ID | Image     | Port(s)       | CPU (%) | Memory usag...   | Mem | Actions |
|--------------------------|---|----------------|--------------|-----------|---------------|---------|------------------|-----|---------|
| <input type="checkbox"/> | ▼ | cityflow-nosql | -            | -         | -             | 0.71%   | 118.22MB / 15.3i |     |         |
| <input type="checkbox"/> |   | cityflow-mon   | d38fe9c96da5 | mongo:7.0 | 27017:27017 ↗ | 0.71%   | 75.82MB / 7.65G  |     |         |
| <input type="checkbox"/> |   | cityflow-mon   | 6571476c7489 | mongo-exp | 8081:8081 ↗   | 0%      | 42.4MB / 7.65GB  |     |         |

1.6. Étape 4 : ouvrir mongo-express dans le navigateur

Ouvrez votre navigateur et allez à : **http://localhost:8081**

Une fenêtre d'authentification basique apparaît. Entrez :

- **Utilisateur** : student
- **Mot de passe** : nosql2025

Vous arrivez sur la page d'accueil de mongo-express. Vous voyez la liste des bases de données existantes : admin, config, local. Ce sont des bases internes à MongoDB — ne les touchez pas. Nous allons créer la nôtre dans la partie suivante.

Mongo Express Database ▾

Mongo Express

Databases

Database Name

+ Create Database

View

admin

Del

View

config

Del

View

local

Del

Server Status

|                     |                               |                       |                     |
|---------------------|-------------------------------|-----------------------|---------------------|
| Hostname            | d38fe9c96da5                  | MongoDB Version       | 7.0.34              |
| Uptime              | 66 seconds                    | Node Version          | 18.20.3             |
| Server Time         | Sat, 30 May 2026 14:52:05 GMT | V8 Version            | 10.2.154.26-node.37 |
|                     |                               |                       |                     |
| Current Connections | 3                             | Available Connections | 100285              |
| Active Clients      | 0                             | Queued Operations     | 0                   |
| Clients Reading     | 0                             | Clients Writing       | 0                   |
| Read Lock Queue     | 0                             | Write Lock Queue      | 0                   |
|                     |                               |                       |                     |
| Disk Flushes        |                               | Last Flush            |                     |
| Time Spent Flushing | ms                            | Average Flush Time    | ms                  |
|                     |                               |                       |                     |
| Total Inserts       | 0                             | Total Queries         | 1                   |
| Total Updates       | 0                             | Total Deletes         | 0                   |

1.7. Étape 5 : commandes utiles pour gérer la stack

# Voir les logs en temps réel

`docker compose logs -f`

# Arrêter la stack (mais garder les données)

`docker compose stop`

# Relancer

`docker compose start`

# Tout arrêter ET supprimer les conteneurs (les données restent dans le volume)

`docker compose down`

# TOUT SUPPRIMER y compris les données (ATTENTION, irréversible)

`docker compose down -v`

### VOTRE TOUR — Validation de la mise en route

Avant de passer à la suite, vérifiez que vous avez accompli ces 5 points :

1. Le dossier `cityflow-nosql` existe sur votre machine.
2. Le fichier `docker-compose.yml` y est, sans erreur de syntaxe.
3. `docker compose ps` montre les deux conteneurs en état **Up**.
4. `http://localhost:8081` ouvre l'interface `mongo-express` après authentification.
5. Vous voyez les 3 bases système (`admin`, `config`, `local`) dans l'interface.

## Partie 2 — Découverte de l'interface mongo-express

### 2.1. Objectif

Avant de créer quoi que ce soit, prenez le temps d'explorer mongo-express pour vous familiariser. Vous allez voir où trouver chaque fonctionnalité — c'est plus facile que d'apprendre par cœur des commandes.

### 2.2. Anatomie de l'interface

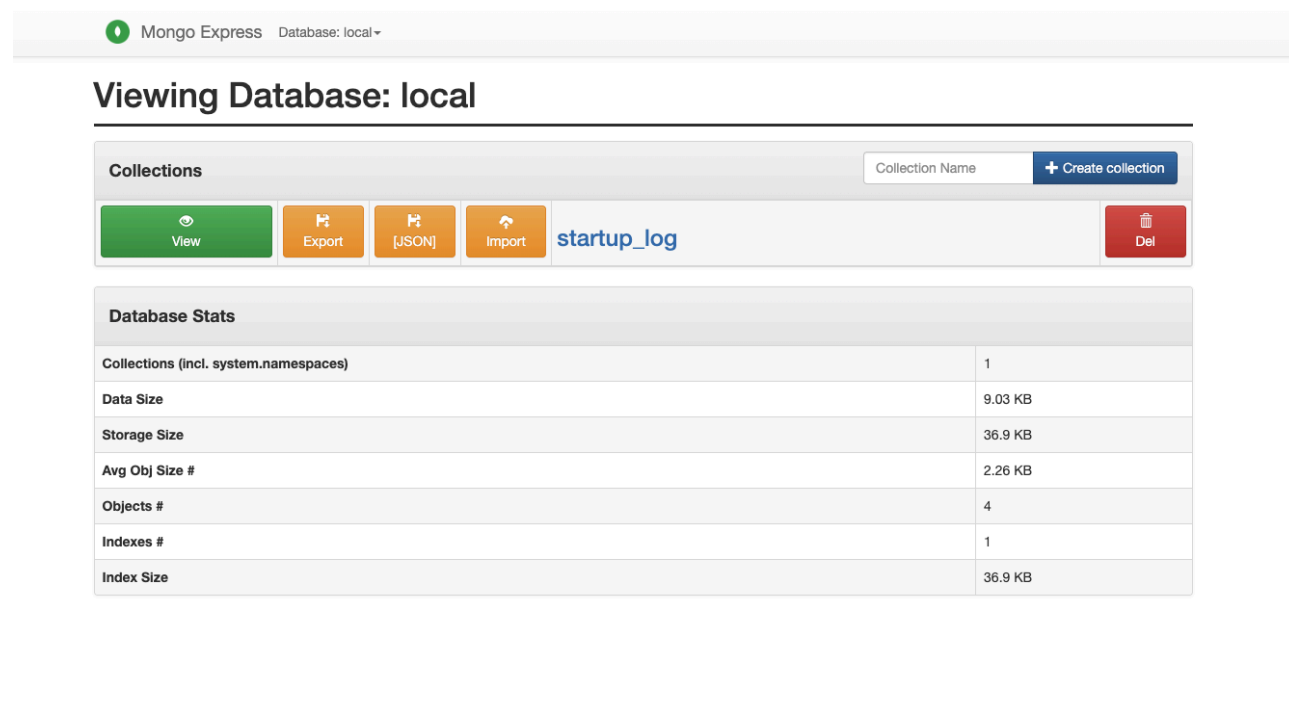
#### Page d'accueil

- **Tableau central** : liste des bases de données existantes
- **Bouton `Create Database`** : pour créer une nouvelle base (on l'utilisera tout à l'heure)
- **Server Status** : informations techniques sur l'état du serveur MongoDB

#### Vue d'une base de données

Quand vous cliquez sur le nom d'une base, vous arrivez sur une page listant ses **collections**. C'est l'équivalent d'une « database » qui contient des « tables » en SQL, sauf qu'ici on parle de **collections** qui contiennent des **documents**. Vous trouverez en haut :

- Le nom de la base et des statistiques (taille, nombre de collections)
- Un bouton Create Collection
- Un bouton Delete Database (à utiliser avec prudence !)



Mongo Express Database: local

### Viewing Database: local

Collections

Collection Name  + Create collection

View Export [JSON] Import startup\_log Del

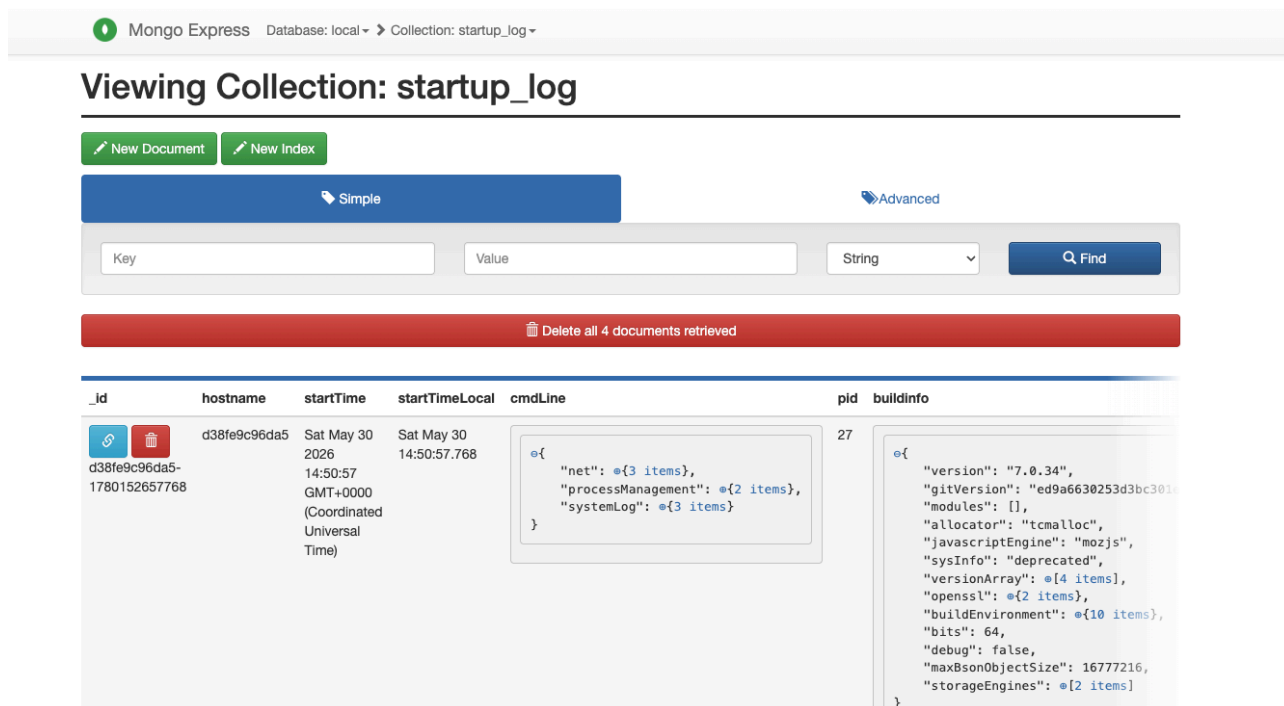
Database Stats

|                                       |         |
|---------------------------------------|---------|
| Collections (incl. system.namespaces) | 1       |
| Data Size                             | 9.03 KB |
| Storage Size                          | 36.9 KB |
| Avg Obj Size #                        | 2.26 KB |
| Objects #                             | 4       |
| Indexes #                             | 1       |
| Index Size                            | 36.9 KB |

#### Vue d'une collection

C'est là où vous passerez le plus de temps. Cette vue est très riche :

- **Onglet Documents** : liste paginée de tous les documents de la collection
- **Onglet Indexes** : gestion des index pour la performance
- **Onglet Stats** : statistiques (taille, nombre, indexation)
- **Bouton `New Document`** : ajouter un document avec un éditeur JSON
- **Bouton `Simple`** : recherche simplifiée par champ
- **Bouton `Advanced`** : projection, tri, limite, skip — équivalent SQL



## 2.3. Exploration libre

### VOTRE TOUR — Exploration

Pendant 10 minutes, explorez l'interface **sans rien modifier**. Cliquez partout :

1. Ouvrez la base admin puis cliquez sur la collection system.version.
2. Observez les documents qu'elle contient (versions internes MongoDB).
3. Retournez à l'accueil, ouvrez config, regardez ses collections.
4. Repérez où se trouvent les boutons : New Document, Search, Indexes, Stats.
5. Notez sur papier les 3 fonctionnalités qui vous intriguent le plus.

**Aucune action de création/modification pendant cette exploration.**

## Partie 3 — Cours : le modèle document

### Vocabulaire MongoDB et correspondance SQL

Pour faciliter la transition depuis le monde relationnel, voici la correspondance des concepts. Attention : la **correspondance n'est pas une équivalence**. Modéliser en MongoDB demande de penser différemment, comme vous le verrez dans la suite

| Concept SQL      | Concept MongoDB | Remarque                                         |
|------------------|-----------------|--------------------------------------------------|
| Database         | Database        | Conteneur de collections                         |
| Table            | Collection      | Conteneur de documents — sans schéma imposé      |
| Row (ligne)      | Document        | Structure JSON-like, hétérogène                  |
| Column (colonne) | Field (champ)   | Présence non obligatoire d'un document à l'autre |

|             |                                    |                                                    |
|-------------|------------------------------------|----------------------------------------------------|
| Primary Key | <code>_id</code>                   | Toujours présent, auto-généré (ObjectId) ou défini |
| JOIN        | <code>\$lookup</code> (agrégation) | Possible mais coûteux et limité — à éviter         |
| Foreign Key | Référence (ObjectId)               | Pas de contrainte d'intégrité côté base            |
| Index       | Index                              | Concept identique, syntaxe différente              |

|        |                                                  |                                       |
|--------|--------------------------------------------------|---------------------------------------|
| INSERT | <code>insertOne</code> / <code>insertMany</code> | Insertion d'un ou plusieurs documents |
| SELECT | <code>find</code>                                | Renvoie un curseur d'itération        |
| UPDATE | <code>updateOne</code> / <code>updateMany</code> | Modification de documents existants   |
| DELETE | <code>deleteOne</code> / <code>deleteMany</code> | Suppression de documents              |

**Le piège mental à éviter : ne raisonnez pas en tables. Un document MongoDB peut contenir ce qui serait dans 3-4 tables SQL différentes. Pensez en entité métier complète plutôt qu'en lignes normalisées.**

### 3.2. Anatomie d'un document

Un document MongoDB est essentiellement un objet JSON enrichi, stocké au format **BSON** (Binary JSON). BSON ajoute à JSON le support de types binaires natifs (entiers, dates, données binaires brutes, identifiants ObjectId), ce qui le rend plus efficace pour le stockage et plus expressif pour les requêtes.

Voici un document complet, qui sera notre fil conducteur :

```
{
  "_id": ObjectId("65f3a8c2b4d8e1f23a4b5c6d"),
  "email": "alice.dupont@example.com",
  "firstName": "Alice",
  "lastName": "Dupont",
  "age": 28,
  "isVerified": true,
  "createdAt": ISODate("2025-09-15T10:30:00Z"),
  "addresses": [
    {
      "type": "home",
      "street": "12 rue de la République",
      "city": "Lyon",
      "zipCode": "69001"
    },
    {
      "type": "work",
      "street": "5 place Bellecour",
      "city": "Lyon",

```

```

    "zipCode": "69002"
  },
  "preferences": {
    "language": "fr",
    "notifications": {
      "email": true,
      "sms": false
    }
  },
  "tags": ["premium", "eco-friendly", "early-adopter"]
}

```

### Observations importantes sur ce document

- **Le champ `\_id`** : présent obligatoirement, c'est l'identifiant unique
- **Les champs simples** : email, firstName, age — comme en SQL
- **Le tableau `addresses`** : un même utilisateur peut avoir plusieurs adresses **dans le même document** — impensable en SQL pur
- **L'objet imbriqué `preferences`** : des préférences elles-mêmes structurées — encore impensable en SQL pur
- **Le tableau de strings `tags`** : un utilisateur peut avoir plusieurs tags sans table de jointure

### 3.3. Les types de données MongoDB (BSON)

MongoDB stocke ses documents au format **BSON** (Binary JSON), qui enrichit JSON avec des types natifs précis. Les principaux types à connaître :

| Type     | Notation       | Exemple                              |
|----------|----------------|--------------------------------------|
| String   | "texte"        | "alice@example.com"                  |
| Integer  | nombre entier  | 28, 1500                             |
| Double   | nombre décimal | 3.14, 19.99                          |
| Boolean  | true / false   | true                                 |
| Date     | ISODate(...)   | ISODate("2025-09-15T10:30:00Z")      |
| ObjectId | ObjectId(...)  | ObjectId("65f3a8c2b4d8e1f23a4b5c6d") |
| Array    | [ ... ]        | ["bike", "scooter"]                  |
| Object   | { ... }        | { "city": "Lyon" }                   |
| Null     | null           | null                                 |

#### Le champ `\_id` et ObjectId

Chaque document doit avoir un `\_id` unique dans sa collection. Si vous ne le fournissez pas, MongoDB en génère un automatiquement avec le type **ObjectId**, un identifiant 12 octets :

- **4 octets** : timestamp Unix de la création — vous pouvez extraire la date depuis un ObjectId !
- **5 octets** : valeur aléatoire unique par machine et processus
- **3 octets** : compteur incrémental pour les insertions dans la même seconde

Conséquence pratique : deux ObjectId générés à la milliseconde près sur deux machines différentes sont quasi-garantis uniques, sans coordination centrale.

## Règles et limites importantes

- Taille maximale d'un document : **16 Mo**. Au-delà, utilisez GridFS pour les gros fichiers.
- Profondeur d'imbrication maximale : **100 niveaux**. Inutile d'y arriver, c'est un signal de mauvaise modélisation.
- Noms de champs : pas de \$ au début, pas de . dans le nom (réservés à MQL).
- Pas de schéma imposé par défaut, MAIS MongoDB supporte le **Schema Validation** depuis la 3.6 —fortement recommandé en production.
- Les noms de champs sont sensibles à la casse : userName≠username.

### 3.4. La règle d'or de la modélisation

**Modélisez selon vos requêtes, pas selon vos données.**

En SQL, on normalise les données en tables logiques, puis on écrit n'importe quelle requête avec JOIN. En MongoDB, c'est l'inverse : on liste les questions que l'application va poser aux données, puis on conçoit les documents pour que chaque question principale tape un seul document si possible.

### 3.5. Embedding vs Référencement

Quand deux entités sont en relation (un utilisateur et ses adresses, un article et ses commentaires, un trajet et son véhicule), vous avez deux options en MongoDB. Comprendre ce choix, c'est maîtriser MongoDB.

#### Option A — Embedding (imbrication)

Les sous-entités sont stockées **directement à l'intérieur** du document parent. C'est ce qu'on a vu avec addresses et preferences dans le document d'Alice.

#### Option B — Référencement (références)

Les sous-entités sont stockées dans une **collection séparée**, et le document parent contient leur identifiant (ObjectId).

```
// Collection users
{
  "_id": ObjectId("u1"),
  "firstName": "Alice",
  "tripIds": [ ObjectId("t1"), ObjectId("t2"), ObjectId("t3") ]
}

// Collection trips
{ "_id": ObjectId("t1"), "userId": "u1", "distance": 12.5, "date": ... }
{ "_id": ObjectId("t2"), "userId": "u1", "distance": 8.2, "date": ... }
```

## Comment choisir ?

### Les 5 critères pour l'embedding

- **Relation 1-1 ou 1-à-peu** : un utilisateur a 1 à 5 adresses, pas 10 000
- **Données toujours lues ensemble** : on n'affiche jamais une adresse seule sans son utilisateur

- **Sous-entités sans existence propre** : une adresse n'a de sens que rattachée à un utilisateur
- **Mises à jour peu fréquentes** : pas modifié 100 fois par minute
- **Volume contrôlé** : le document ne risque pas de dépasser 16 Mo (la limite MongoDB)

#### Les 5 critères pour le référencement

- **Relation N-N ou 1-à-beaucoup** : un utilisateur a des milliers de trajets
- **Sous-entités avec existence propre** : un véhicule existe indépendamment
- **Données interrogées séparément** : on veut requêter les véhicules sans charger les users
- **Mises à jour fréquentes** : modifier un véhicule sans réécrire ses trajets
- **Volume potentiellement explosif**

**Test mental rapide** : posez-vous la question « est-ce que ces données ont du sens indépendamment du parent, et est-ce qu'il peut y en avoir énormément ? » Si oui aux deux → référencement. Sinon → embedding.

#### Exemple :

Appliquons ces critères au domaine de CityFlow pour produire une modélisation cohérente.

| Relation                     | Stratégie                 | Justification                                                                   |
|------------------------------|---------------------------|---------------------------------------------------------------------------------|
| <b>User → Addresses</b>      | Embedding                 | Peu d'adresses par user (1-3), toujours lues ensemble, pas d'existence propre   |
| <b>User → Preferences</b>    | Embedding                 | Relation 1-1, lecture systématique avec le profil                               |
| <b>User → Trips</b>          | Référencement             | 1 user a potentiellement des milliers de trajets, lus séparément                |
| <b>Trip → User</b>           | Référencement (userId)    | Le trajet stocke l'ID de l'utilisateur ; on n'embarque pas l'objet user complet |
| <b>Trip → Vehicle</b>        | Référencement (vehicleId) | Un véhicule existe indépendamment, est réutilisé pour de nombreux trajets       |
| <b>Trip → Steps (étapes)</b> | Embedding                 | Les étapes sont propres au trajet et lues ensemble avec lui                     |
| <b>Vehicle → Maintenance</b> | Référencement             | Une voiture peut avoir des centaines d'entrées de maintenance dans sa vie       |

#### Le piège classique : copier son schéma SQL

Le réflexe naturel quand on vient du SQL est de créer une collection par table. C'est la **migration aveugle** mentionnée au chapitre Fondamentaux, et c'est une erreur. Voici un exemple concret pour le comprendre.

## Mauvaise modélisation (copie SQL)

```
// Une collection par table, comme en SQL
collection: users → { _id, name, email }
collection: addresses → { _id, userId, type, city, zip }
collection: phoneNumbers → { _id, userId, type,
number }

// Pour afficher un profil utilisateur, il faut 3 requêtes :
db.users.findOne({_id: "u1"})
db.addresses.find({userId: "u1"})
db.phoneNumbers.find({userId: "u1"})
// → 3 allers-retours réseau, pas de cohérence atomique
```

## Bonne modélisation (orientée document)

```
// Tout dans le document
user
collection: users → {
  _id, name, email,
  addresses: [...],
  phoneNumbers: [...]
}

// Pour afficher un profil utilisateur : UNE requête
db.users.findOne({_id: "u1"})
// → 1 aller-retour réseau, lecture atomique, données
cohérentes
```

*Cette différence n'est pas anecdotique : elle peut diviser par 3 ou 4 la latence de vos requêtes les plus fréquentes. C'est tout l'intérêt d'utiliser MongoDB plutôt qu'un SQL relationnel avec colonnes JSON.*

## Partie 4 — Exercices de modélisation papier

### 4.1. Pourquoi faire de la modélisation sur papier ?

C'est tentant de sauter directement à la création dans mongo-express. **Résistez à cette tentation.** Modéliser sur papier vous force à réfléchir aux trade-offs sans la facilité de corriger en cliquant. C'est exactement ce qu'on vous demandera en entretien d'embauche et en réunion d'équipe.

Pour chaque exercice, prenez une feuille blanche et notez :

- Le nom de chaque collection
- Les champs principaux de chaque document, avec leur type
- Pour chaque relation : choix entre **embedding** ou **référencement**
- **Une justification écrite** pour chaque choix (« je choisis l'embedding parce que... »)

#### VOTRE TOUR — Exercice 1 : Fiche utilisateur de réseau social

Modélisez le profil d'un utilisateur d'un réseau social type LinkedIn. Le profil doit permettre de stocker :

- Les informations personnelles (nom, prénom, email, photo, bio)
- Les coordonnées (téléphones, adresses postales, liens vers réseaux sociaux)
- Les compétences techniques (avec niveau pour chacune : débutant/intermédiaire/expert)
- L'historique professionnel (les postes occupés, entreprises, dates)
- Les paramètres de notifications (email oui/non, sms oui/non, etc.)

##### Travail demandé :

1. Combien de collections allez-vous créer ? Pourquoi ?
2. Pour chaque sous-élément, choisissez embedding ou référencement.
3. Justifiez **chaque** choix sur papier.
4. Estimez la taille moyenne d'un document utilisateur (en supposant 10 ans de carrière, 5 entreprises, 20 compétences).

#### Exercice 2 : Blog avec commentaires

Modélisez une plateforme de blog avec :

- Des **articles** (titre, contenu, date, auteur, tags, image de couverture)
- Des **auteurs** (nom, bio, photo, liste de leurs articles publiés)
- Des **commentaires** sur chaque article (auteur du commentaire, contenu, date)
- Un système de **likes** sur les articles (qui a liké, quand)
- Des **catégories** thématiques

**Question difficile** : un article populaire peut avoir 1000+ commentaires et 10 000+ likes. Comment gérez-vous ces deux cas ? Embedding ou référencement ? Justifiez précisément.

##### Travail demandé :

1. Tracez les collections et leurs relations.
2. Pour les commentaires et les likes, faites un choix argumenté.
3. Anticipez : que devient votre modélisation si l'article fait \*buzz\* avec 100 000 commentaires ?

#### Exercice 3 : E-commerce

Modélisez une boutique en ligne avec :

- Des **produits** (nom, prix, description, stock, catégories, images, attributs variables comme taille/couleur)

- Des **clients** (informations, adresses de livraison multiples, moyens de paiement)
- Des **commandes** (lignes de produits, total, statut, dates, adresse de livraison)
- Un **panier** (en cours, pas encore commandé)
- Des **avis** sur les produits (note, commentaire, auteur)

**Questions clés :**

1. Quand un client passe commande, faut-il copier les détails du produit dans la commande, ou juste référencer le produit ?
2. Le panier est-il une collection à part ou un champ du client ?
3. Comment gérer le fait que le **prix d'un produit peut changer** entre le moment où on le met au panier et le moment où on commande ?

## Partie 5 — Création et insertion guidée dans mongo-express

### 5.1. Objectif

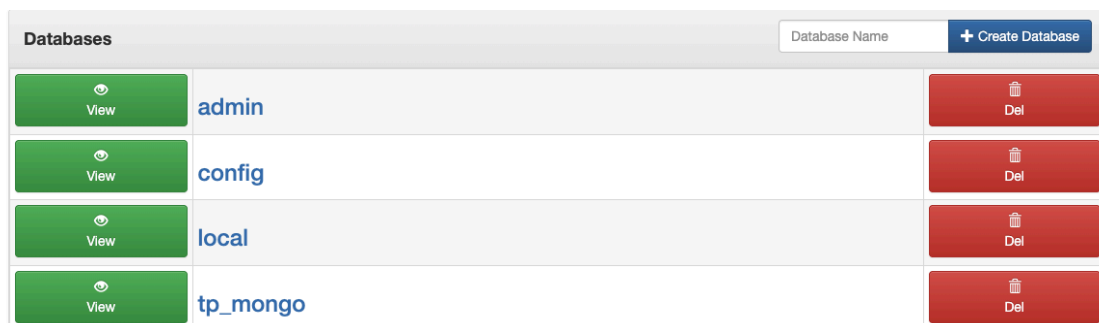
Vous allez maintenant matérialiser vos modélisations dans MongoDB via mongo-express. L'idée : que vous voyez par l'expérience comment se traduit votre conception papier. Vous allez créer une mini-base, insérer des documents, et observer le résultat.

### 5.2. Étape 1 : créer la base de données

Dans mongo-express, sur la page d'accueil :

- Repérez le champ **Create Database** en bas de la liste des bases
- Tapez le nom : **tp\_mongo**
- Cliquez sur **Create Database**

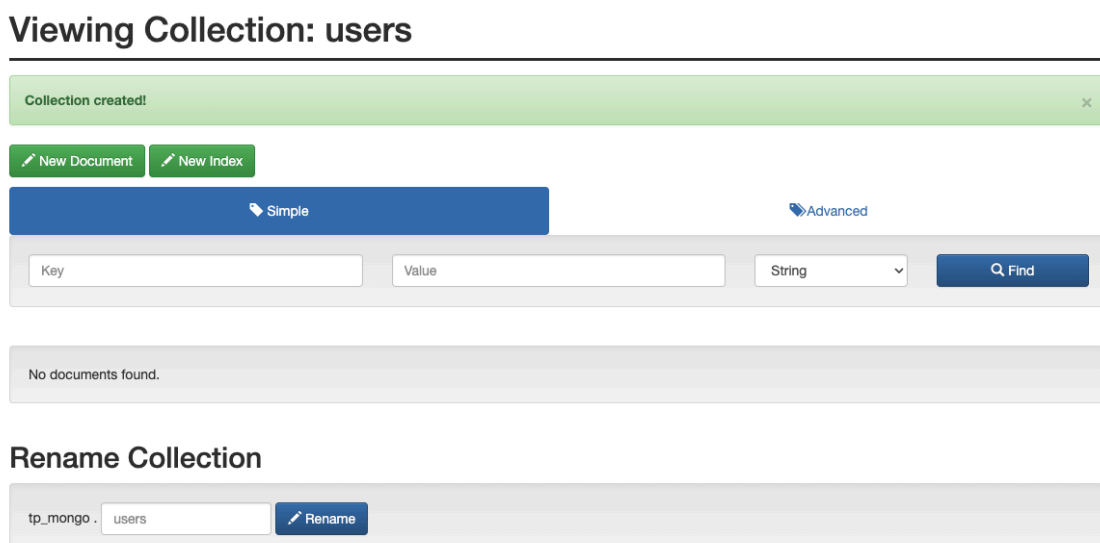
*Astuce : MongoDB ne crée pas vraiment la base tant qu'aucune collection ni document n'y est inséré. Mais mongo-express affiche déjà votre base, ce qui est suffisant pour la suite.*



### 5.3. Étape 2 : créer une collection

Cliquez sur la base tp\_mongo que vous venez de créer. Vous arrivez sur sa page :

- Cliquez sur le bouton **Create Collection**
- Tapez le nom : **users**
- Validez



### 5.4. Étape 3 : insérer votre premier document

Cliquez sur la collection users. Vous arrivez sur la vue détaillée. Cliquez sur **New Document** :

Un éditeur JSON s'ouvre, prérempli avec un squelette. **Effacez-le entièrement** et remplacez par :

```
{
  "email": "alice.dupont@example.com",
  "firstName": "Alice",
  "lastName": "Dupont",
  "age": 28,
  "isVerified": true,
  "createdAt": ISODate("2025-10-01T08:00:00Z"),
  "addresses": [
    {
      "type": "home",
      "street": "12 rue de la République",
      "city": "Lyon",
      "zipCode": "69001"
    }
  ],
  "tags": ["premium", "early-adopter"]
}
```

Cliquez sur **`Save`**. Si tout est bien, vous voyez le document apparaître dans la liste, **avec un `\_id` généré automatiquement** par MongoDB.

### Viewing Collection: users

### 5.5. Étape 4 : observer le document inséré

Une fois le document inséré, repérez :

- Le **`\_id`** auto-généré (un ObjectId)
- Les **boutons d'action** à droite : View, Edit, Duplicate, Delete
- Cliquez sur **`View`** pour voir le document formaté en lecture seule
- Cliquez sur **`Edit`** pour modifier — observez que MongoDB a stocké la date en ISODate

### VOTRE TOUR — Insertions multiples

Insérez **5 utilisateurs supplémentaires** dans la collection users. Variez les profils pour rendre les requêtes intéressantes par la suite :

- Au moins **2 villes différentes** (Lyon, Villeurbanne, Vénissieux, etc.)
- Au moins **2 utilisateurs sans le champ `isVerified`** (pour voir le schéma flexible en action)
- Au moins **1 utilisateur avec 2 adresses** dans son tableau
- Au moins **1 utilisateur avec un champ `preferences`** que personne d'autre n'a
- Variez les âges (de 18 à 60 ans) pour les requêtes de filtre futures

Vous pouvez utiliser le bouton **`Duplicate`** pour partir d'un document existant et le modifier.

**À la fin** : vous devez avoir 6 documents dans la collection users.

### 5.6. Observation : le schéma flexible en action

Une fois vos 6 documents insérés, observez la liste. Vous remarquerez :

- **Certains documents ont le champ `isVerified`**, d'autres non — MongoDB ne s'en plaint pas
- **Le champ `preferences`** n'existe que sur un document — c'est valide
- **Le tableau `addresses`** a 1 ou 2 éléments selon le document — c'est valide

**C'est l'essence du modèle document : chaque document est indépendant et peut avoir sa propre structure. Ce qui serait impossible en SQL (où chaque ligne d'une table a exactement les mêmes colonnes) est la norme en MongoDB.**

**Attention : cette flexibilité a un coût. Sans Schema Validation activée (à voir plus tard), n'importe qui peut insérer n'importe quoi. En production, on impose souvent un schéma minimal pour éviter les dérives.**

### 5.7. Modification et suppression

Testez les actions sur vos documents :

- **Edit** : modifiez l'âge d'Alice, sauvegardez, vérifiez la mise à jour
- **Duplicate** : créez une copie d'un document, **modifiez l'email pour le rendre unique**, sauvegardez
- **Delete** : supprimez un document de test (pas Alice !)

⚠ **Attention au champ `\_id`** : si vous dupliquez et essayez de garder le même `\_id`, mongo-express lèvera une erreur. C'est la seule contrainte d'unicité par défaut. Pour les autres champs (email par exemple), il faut créer un **index unique** que nous verrons en partie 9.

## Partie 6 — Cours : requêtes et opérateurs MQL

### 6.1. Anatomie d'une requête MongoDB

Une requête MongoDB se compose de deux parties principales :

- **Le filtre** : un objet JSON décrivant les conditions (équivalent du WHERE SQL)
- **La projection** : un objet JSON listant les champs à renvoyer (équivalent du SELECT champs SQL)

Dans mongo-express, deux modes de recherche :

- **Simple** : champ par champ, sans opérateurs avancés
- **Advanced** : on saisit un JSON complet avec opérateurs MQL

### 6.2. Les opérateurs de comparaison

| Opérateur | Sens                  | Exemple de filtre                                   |
|-----------|-----------------------|-----------------------------------------------------|
| \$eq      | égal                  | { "age": { "\$eq": 28 } }                           |
| \$ne      | différent             | { "city": { "\$ne": "Lyon" } }                      |
| \$gt      | strictement supérieur | { "age": { "\$gt": 30 } }                           |
| \$gte     | supérieur ou égal     | { "age": { "\$gte": 18 } }                          |
| \$lt      | strictement inférieur | { "age": { "\$lt": 30 } }                           |
| \$lte     | inférieur ou égal     | { "age": { "\$lte": 65 } }                          |
| \$in      | dans une liste        | { "addresses.city": { "\$in": ["Lyon", "Paris"] } } |
| \$nin     | pas dans une liste    | { "status": { "\$nin": ["banned"] } }               |

### 6.3. Les opérateurs logiques

Pour combiner plusieurs conditions :

```
// AND implicite (le plus courant)
{ "age": { "$gte": 18 }, "addresses.city": "Lyon" }

// AND explicite (équivalent)
{ "$and": [
  { "age": { "$gte": 18 } },
  { "addresses.city": "Lyon" }
]}

// OR
{ "$or": [
  { "age": { "$lt": 18 } },
  { "isVerified": true }
]}

// NOT
{ "age": { "$not": { "$gte": 18 } } }

// NOR (aucune des conditions vraie)
{ "$nor": [
```

```
{ "isVerified": false },  
  { "addresses.city": "Lyon" }  
}]
```

## 6.4. Les opérateurs sur les champs et les tableaux

Pour interroger les structures complexes :

```
// Existence d'un champ  
{ "preferences": { "$exists": true } }  
  
// Type d'un champ  
{ "age": { "$type": "int" } }  
  
// Un élément du tableau correspond  
{ "tags": "premium" } // au moins un tag = "premium"  
  
// Tous les éléments correspondent  
{ "tags": { "$all": ["premium", "eco-friendly"] } }  
  
// Au moins un élément du tableau vérifie plusieurs critères  
{ "addresses": { "$elemMatch": { "city": "Lyon", "type": "home" } } }  
  
// Tableau de taille exacte  
{ "addresses": { "$size": 2 } }  
  
// Notation pointée pour accéder aux champs imbriqués  
{ "preferences.language": "fr" }  
{ "addresses.city": "Lyon" }
```

**Notation pointée : c'est l'outil le plus utile de MQL. Vous accédez aux champs imbriqués comme en JavaScript avec un point. Pour un tableau d'objets, MongoDB vérifie si au moins un élément du tableau correspond.**

## 6.5. Tri, pagination, projection

### Tri

```
// Tri ascendant  
{ "age": 1 }  
  
// Tri descendant  
{ "createdAt": -1 }  
  
// Tri multi-critères  
{ "addresses.city": 1, "age": -1 }
```

### Pagination (Skip + Limit)

Skip = sauter N résultats, Limit = en renvoyer N maximum. Pour la page 3 avec 10 résultats par page : skip: 20, limit: 10.

### Projection (sélection des champs)

```
// Inclure uniquement certains champs (1 = inclure)
{ "firstName": 1, "email": 1, "_id": 0 }

// Exclure certains champs (0 = exclure)
{ "addresses": 0, "preferences.notifications": 0 }

// Note : on ne mélange pas inclusions et exclusions
// SAUF pour _id qu'on peut toujours exclure
```

## Partie 7 — TP : requêtes pratiques à produire

### 7.1. Objectif et méthode

Vous avez vu les opérateurs en théorie. Maintenant, vous allez **écrire vos propres requêtes** sur vos données. Pour chaque demande ci-dessous :

- Réfléchissez à la structure de la requête (filtre + projection + tri)
- Saisissez-la dans mongo-express via l'option **Advanced** de la barre de recherche
- Observez le résultat — si zéro résultat, c'est peut-être normal (vérifiez vos données)
- **Notez la requête finale** dans un fichier mes\_requetes.md pour les exercices

### 7.2. Comment utiliser le mode Advanced de mongo-express

Sur la page de votre collection users, en haut, repérez la barre de recherche. Cliquez sur le bouton **Advanced** à droite. Trois champs apparaissent :

- **Query** : le filtre JSON (équivalent du WHERE)
- **Projection** : les champs à renvoyer
- **Sort** : le tri
- **Limit / Skip** : pagination

**Important** : les valeurs doivent être saisies en **JSON strict**. Donc :

- Les noms de champs sont **entre guillemets** : "age" et pas age
- Les chaînes sont **entre guillemets doubles** : "Lyon"
- Les booléens sans guillemets : true, false
- Pas de virgule finale dans les objets

Si mongo-express affiche Invalid JSON, c'est presque toujours une virgule en trop ou un guillemet manquant.

### 7.3. Vos premières requêtes guidées

Pour démarrer, voici **3 exemples résolus** que vous pouvez recopier pour comprendre la syntaxe :

#### Exemple 1 : tous les utilisateurs majeurs (18 ans et plus)

```
// Query
{ "age": { "$gte": 18 } }

// Projection (optionnelle)
{ "firstName": 1, "lastName": 1, "age": 1, "_id": 0 }
```

#### Exemple 2 : utilisateurs de Lyon

```
// Query
{ "addresses.city": "Lyon" }
```

#### Exemple 3 : utilisateurs vérifiés OU ayant le tag premium

```
// Query
{ "$or": [
```

```
{ "isVerified": true },  
  { "tags": "premium" }  
}]}
```

## VOTRE TOUR — Requêtes à produire

Pour chacune des demandes suivantes, **écrivez la requête MongoDB correspondante** dans mongo-express, et notez-la dans votre fichier mes\_requetes.md. **Aucune requête ne vous est donnée toute faite.**

### Niveau 1 — Filtres simples

1. Trouvez tous les utilisateurs entre 25 et 35 ans (inclus aux deux bornes).
2. Trouvez les utilisateurs dont l'email contient example.com (indice : opérateur \$regex).
3. Trouvez les utilisateurs qui ont **exactement 2 adresses**.
4. Trouvez les utilisateurs qui **n'ont pas** le champ preferences.

### Niveau 2 — Combinaisons

5. Trouvez les utilisateurs habitant Lyon **OU** Villeurbanne, et de plus de 25 ans.
6. Trouvez les utilisateurs qui ont au moins une adresse de type home à Lyon.
7. Trouvez les utilisateurs qui ont les tags premium **ET** eco-friendly ensemble.

### Niveau 3 — Tri et pagination

8. Affichez les 3 utilisateurs les plus âgés, en ne renvoyant que firstName, age et email.
9. Affichez les utilisateurs triés par ville (alphabétique), puis par âge décroissant.
10. Implémentez une pagination : page 2 avec 2 utilisateurs par page, triés par createdAt.

### Niveau 4 — Bonus pour les rapides

11. Trouvez les utilisateurs vérifiés âgés de moins de 30 ans, n'habitant pas Lyon.
12. Trouvez les utilisateurs dont au moins un tag commence par la lettre 'p' (indice : \$regex dans un tableau).

## 7.4. Points d'arrêt et conseils

### Si vous bloquez sur une requête :

1. Vérifiez votre **syntaxe JSON** : guillemets, virgules.
2. Vérifiez que vos **données contiennent bien le cas** que vous cherchez — si personne dans votre base n'a 25-35 ans, vous aurez zéro résultat même avec une requête correcte.
3. **Décomposez** : commencez par un filtre simple, puis ajoutez les conditions une par une.
4. Consultez le cheat sheet en annexe à la fin du document.
5. Demandez à un voisin **avant** de demander au formateur — souvent l'erreur est évidente vue de l'extérieur.

## Partie 8 — Cours : pipeline d'agrégation et index

### 8.1. Pourquoi a-t-on besoin d'agrégation ?

Les requêtes find() vues jusqu'ici sont parfaites pour récupérer des documents existants. Mais que faites-vous quand vous voulez :

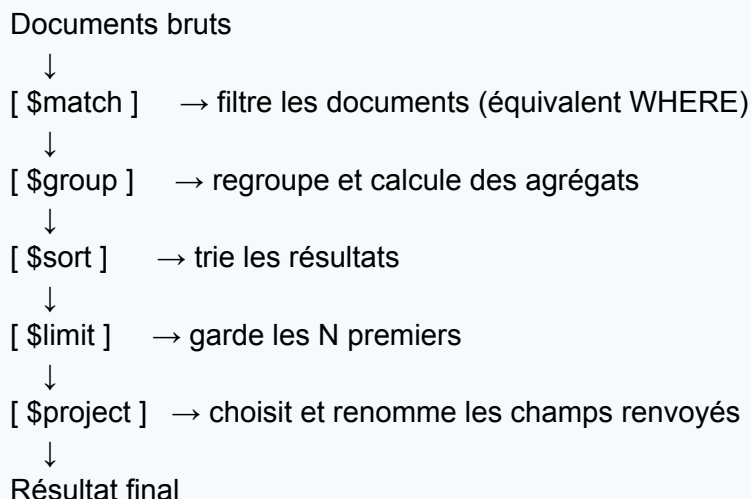
- Compter le nombre d'utilisateurs par ville ?
- Calculer l'âge moyen des utilisateurs vérifiés ?

- Trouver les 5 villes les plus représentées ?
- Calculer un chiffre d'affaires par mois ?

C'est ce que l'**aggregation framework** de MongoDB permet de faire. C'est l'équivalent de GROUP BY + fonctions d'agrégation SQL, mais beaucoup plus puissant.

## 8.2. Le principe du pipeline

Un pipeline d'agrégation est une **succession d'étapes** qui transforment progressivement les données. C'est exactement comme les pipes Unix (cmd1 | cmd2 | cmd3).



## 8.3. Les étapes principales

| Étape     | Rôle                                      | Équivalent SQL            |
|-----------|-------------------------------------------|---------------------------|
| \$match   | Filtrer les documents                     | WHERE                     |
| \$group   | Regrouper et calculer des agrégats        | GROUP BY                  |
| \$sort    | Trier                                     | ORDER BY                  |
| \$limit   | Limiter                                   | LIMIT                     |
| \$skip    | Sauter N résultats                        | OFFSET                    |
| \$project | Sélectionner/renommer/calculer des champs | SELECT col AS             |
| \$lookup  | Jointure avec une autre collection        | LEFT JOIN                 |
| \$unwind  | Déplier un tableau en plusieurs documents | (pas d'équivalent direct) |
| \$count   | Compter le nombre de documents            | COUNT(*)                  |

## 8.4. Les accumulateurs dans \$group

Dans \$group, on utilise des **accumulateurs** pour calculer des agrégats :

| Accumulateur | Calcul | Exemple                                               |
|--------------|--------|-------------------------------------------------------|
| \$sum        | Somme  | { "\$sum": "\$price" } ou { "\$sum": 1 } pour compter |

|            |                                   |                            |
|------------|-----------------------------------|----------------------------|
| \$avg      | Moyenne                           | { "\$avg": "\$age" }       |
| \$min      | Minimum                           | { "\$min": "\$createdAt" } |
| \$max      | Maximum                           | { "\$max": "\$price" }     |
| \$first    | Premier élément du groupe         | { "\$first": "\$name" }    |
| \$last     | Dernier élément du groupe         | { "\$last": "\$name" }     |
| \$push     | Tous les éléments dans un tableau | { "\$push": "\$name" }     |
| \$addToSet | Éléments uniques dans un tableau  | { "\$addToSet": "\$city" } |

## 8.5. Exemples concrets

### Exemple 1 : compter les utilisateurs par ville

```
[
  { "$unwind": "$addresses" },
  { "$group": {
    "_id": "$addresses.city",
    "userCount": { "$sum": 1 }
  }},
  { "$sort": { "userCount": -1 } }
]
```

### Exemple 2 : âge moyen et nombre d'utilisateurs vérifiés

```
[
  { "$match": { "isVerified": true } },
  { "$group": {
    "_id": null,
    "verifiedCount": { "$sum": 1 },
    "avgAge": { "$avg": "$age" },
    "youngest": { "$min": "$age" },
    "oldest": { "$max": "$age" }
  }}
]
```

### Exemple 3 : top 3 des tags les plus utilisés

```
[
  { "$unwind": "$tags" },
  { "$group": {
    "_id": "$tags",
    "count": { "$sum": 1 }
  }},
  { "$sort": { "count": -1 } },
  { "$limit": 3 }
]
```

## 8.6. Les index : pourquoi et comment

Sans index, MongoDB doit scanner tous les documents pour répondre à une requête (**collection scan**). C'est rapide sur 100 documents, mais devient catastrophique sur 100 millions. Avec un bon index, MongoDB localise les documents en  $O(\log n)$ .

### Les types d'index principaux

| Type         | Usage                                                              |
|--------------|--------------------------------------------------------------------|
| Single field | Index sur un seul champ : { email: 1 }                             |
| Compound     | Index sur plusieurs champs (l'ordre compte) : { city: 1, age: -1 } |
| Unique       | Garantit l'unicité d'un champ : email, username                    |
| Text         | Recherche full-text dans des champs texte                          |
| TTL          | Suppression automatique après expiration (sessions, tokens)        |
| Geospatial   | Recherches géographiques (à proximité de...)                       |

### La règle ESR pour les index composés

Quand on crée un index sur plusieurs champs, l'ordre des champs dans l'index compte. La règle **ESR** (Equality, Sort, Range) recommandée par MongoDB Inc. :

- **E — Equality** : placez en premier les champs utilisés en égalité (city = "Lyon")
- **S — Sort** : ensuite les champs utilisés pour trier
- **R — Range** : enfin les champs utilisés dans des intervalles (age >= 18)

### Le coût des index

- **Bénéfice** : lectures rapides
- **Coût** : insertions et mises à jour plus lentes (il faut maintenir l'index)
- **Coût** : espace disque supplémentaire

**Règle d'or : indexez les champs utilisés dans les filtres fréquents et les tris. Ne pas indexer les champs très peu sélectifs (par exemple gender avec 2 valeurs possibles) ou les champs jamais utilisés dans les filtres.**

## Partie 9 — TP : agrégation et index sur vos données

### 9.1. Utiliser l'Aggregation Builder de mongo-express

mongo-express expose l'aggregation framework via un éditeur dédié. Dans la vue d'une collection :

- Cherchez le bouton **`Aggregate`** (selon la version)
- Saisissez votre pipeline sous forme de tableau JSON [ {étape1}, {étape2}, ... ]
- Lancez et observez les résultats

#### VOTRE TOUR — Pipelines d'agrégation

Sur votre collection users, **écrivez et testez** les pipelines suivants. Notez chaque pipeline dans votre fichier mes\_requetes.md avec le résultat obtenu. Pour cette fois vous utilisez le terminal mongosh.

##### Niveau 1 — Agrégations simples

1. Comptez le nombre total d'utilisateurs dans votre collection.
2. Calculez l'âge moyen, le plus jeune et le plus âgé de votre base.
3. Comptez combien d'utilisateurs sont vérifiés (et combien ne le sont pas).

##### Niveau 2 — Regroupements

4. Comptez le nombre d'utilisateurs par ville (utilisez \$unwind sur addresses).
5. Trouvez l'âge moyen par ville, trié par âge moyen décroissant.
6. Identifiez les 3 tags les plus utilisés et le nombre d'utilisateurs pour chacun.

##### Niveau 3 — Pipelines complexes

7. Pour chaque ville, listez les firstName de tous les utilisateurs qui y habitent (utilisez \$push ou \$addToSet).
8. Calculez la **proportion d'utilisateurs vérifiés par ville** (champ calculé : verifiedCount / totalCount).
9. Trouvez les utilisateurs qui ont le plus grand nombre d'adresses (\$project pour ajouter un champ addressCount).

### 9.2. Créer et tester des index

Dans la vue de la collection users, cliquez sur l'onglet **`Indexes`**. Vous voyez la liste des index actuels — il n'y en a qu'un par défaut : l'index sur \_id. Ajoutons-en.

#### Créer un index simple

- Cliquez sur **`Create Index`**
- Dans le champ Key, saisissez : { "email": 1 }
- Cochez la case **Unique** (un email ne doit apparaître qu'une seule fois)
- Validez

Vérifiez que l'index apparaît dans la liste. **Essayez maintenant d'insérer un document avec un email déjà existant** dans la collection. mongo-express devrait refuser l'opération avec une erreur de duplicate key.

#### VOTRE TOUR — Stratégie d'indexation

En regardant les requêtes que vous avez écrites dans la partie 7, **identifiez et créez les index qui amélioreraient les performances**.

**Travail demandé :**

1. Listez les champs **les plus utilisés en filtre** dans vos requêtes.
2. Listez les champs **les plus utilisés en tri**.
3. Proposez **au moins 3 index** pertinents (un simple, un composé, un éventuellement unique).
4. Créez-les dans mongo-express via l'onglet Indexes.
5. Pour chaque index créé, **justifiez son existence** dans votre fichier mes\_requetes.md.

**Question bonus :** quel serait l'index idéal pour la requête « utilisateurs habitant Lyon, âgés de 18 à 30 ans, triés par createdAt décroissant » ? Appliquez la règle ESR.

## Partie 10 — Projet fil rouge CityFlow

### 10.1. Objectif

Vous avez maintenant toutes les compétences pour démarrer la partie MongoDB du projet CityFlow. Cette partie est **collective** : faites-la avec votre équipe projet (3-4 personnes). Le résultat alimentera directement votre repo Git.

### 10.2. Rappel du contexte CityFlow

CityFlow est une plateforme de mobilité urbaine multimodale (covoiturage, transports, vélos). Pour la partie MongoDB, vous devez modéliser et alimenter trois collections principales :

- **users** : les utilisateurs de la plateforme
- **trips** : les trajets effectués
- **vehicles** : les véhicules disponibles (vélos, scooters, voitures)

### 10.3. Les user stories à supporter

Votre modélisation doit permettre de répondre aux 4 user stories suivantes :

| ID    | User Story                                                                                                                                        |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| US-M1 | En tant qu'utilisateur, je veux pouvoir consulter mon profil et l'historique de mes 10 derniers trajets.                                          |
| US-M2 | En tant qu'administrateur, je veux pouvoir lister tous les véhicules d'un type donné disponibles dans un arrondissement.                          |
| US-M3 | En tant qu'analyste, je veux pouvoir obtenir des statistiques agrégées : nombre de trajets par jour, distance moyenne, top 5 des conducteurs.     |
| US-M4 | En tant que développeur, je veux pouvoir effectuer une recherche full-text sur les commentaires laissés par les utilisateurs après leurs trajets. |

### 10.4. Étape 1 — Modélisation papier

#### VOTRE TOUR — Modélisation collective

**En équipe**, sur une feuille blanche ou un tableau, modélisez les 3 collections en répondant à ces questions pour chaque collection :

1. Quels sont les champs principaux ?
2. Quels sont les sous-objets et tableaux ?
3. Pour chaque relation entre collections, embedding ou référencement ? **Justifiez.**
4. Quelles sont les requêtes principales que cette collection devra servir (en regardant les US) ?

#### Points de réflexion à débattre dans l'équipe :

- Un trajet doit-il **embarquer** les infos du véhicule ou juste le **référencer** ?
- Quelles **dénormalisations** justifient-elles dans le contexte de CityFlow ?
- Comment supportez-vous la recherche full-text de US-M4 ? (indice : \$text + index text)

**Choisissez un rapporteur** qui présentera le résultat au formateur en 2 minutes.

## 10.5. Étape 2 — Création des collections et des jeux de données

### VOTRE TOUR — Création de la base CityFlow

Dans mongo-express :

1. **Créez la base `cityflow`** (à côté de votre tp\_mongo).
2. **Créez les 3 collections** : users, trips, vehicles.
3. **Insérez au moins** :
  - 15 utilisateurs avec profils variés (âges, villes, préférences)
  - 30 trajets répartis sur les 30 derniers jours
  - 10 véhicules de types variés (au moins 3 vélos, 3 scooters, 3 voitures)

#### Contraintes pour rendre les requêtes intéressantes :

- Utilisez des **vraies villes lyonnaises** : Lyon 1er-9e, Villeurbanne, Vénissieux, etc.
- Variez les **dates** des trajets pour avoir des statistiques par jour intéressantes
- Ajoutez des **commentaires libres** aux trajets pour tester la recherche full-text
- Assurez-vous que **certain utilisateurs ont 5+ trajets** pour le top 5
- Les trajets doivent référencer des `userId` et `vehicleId` qui existent réellement

## 10.6. Étape 3 — Implémenter les 4 user stories

### VOTRE TOUR — Les requêtes des user stories

Pour chaque user story, **écrivez la requête MongoDB** (find ou aggregate selon ce qui convient), testez-la dans mongo-express et notez-la avec sa sortie dans un fichier `queries/mongodb-queries.md` du repo CityFlow.

#### US-M1 : profil + 10 derniers trajets

→ Une ou deux requêtes ? Discutez en équipe puis implémentez.

#### US-M2 : véhicules d'un type donné dans un arrondissement

→ Filtre composé type + location. Avec ou sans index ?

#### US-M3 : statistiques globales

→ Pipeline d'agrégation à plusieurs étapes. Listez les étapes avant d'écrire le JSON.

#### US-M4 : recherche full-text

→ Vous devez d'abord créer un **index text** sur le champ de commentaires, puis utiliser l'opérateur `$text` avec `$search`. Cherchez la documentation MongoDB sur l'index text.

## 10.7. Étape 4 — Documenter et commiter

Pour finaliser cette séance et alimenter votre projet CityFlow :

- Dans votre repo Git, créez un fichier `docs/modelisation-mongodb.md` avec votre modélisation et vos justifications
- Créez un fichier `seed/mongodb/init.js` (script d'insertion) que vous reproduirez plus tard automatiquement — ce TP a été manuel, mais à terme on automatise
- Notez les 4 user stories résolues dans `queries/mongodb-queries.md`
- Justifiez les **index créés** dans la documentation
- Faites un commit Git avec un message clair : `feat(mongo): modélisation initiale et seed`

**À ce stade, vous avez :**

- Une base MongoDB fonctionnelle pour CityFlow
- 3 collections cohérentes avec données réalistes
- 4 user stories implémentées et documentées
- Des index justifiés
- Un livrable Git versionné

**C'est la moitié de votre note de projet CityFlow pour la partie MongoDB !**

## Annexe A — Cheat sheet mongo-express

### A.1. Connexion et bases

| Action                 | Où dans mongo-express ?                           |
|------------------------|---------------------------------------------------|
| Se connecter           | http://localhost:8081 (auth: student / nosql2025) |
| Créer une base         | Page d'accueil → champ Create Database            |
| Supprimer une base     | Vue de la base → bouton Delete Database (rouge)   |
| Lister les collections | Cliquer sur une base → vue par défaut             |
| Créer une collection   | Vue de la base → bouton Create Collection         |

### A.2. Documents

| Action                  | Où dans mongo-express ?                            |
|-------------------------|----------------------------------------------------|
| Voir tous les documents | Vue de la collection → onglet par défaut           |
| Ajouter un document     | Bouton New Document → coller un JSON               |
| Modifier un document    | Bouton Edit à droite d'un document                 |
| Dupliquer               | Bouton Duplicate à droite — utile pour seed rapide |
| Supprimer               | Bouton Delete à droite (avec confirmation)         |
| Vue lecture seule       | Bouton View à droite                               |

### A.3. Requêtes

| Action                   | Où dans mongo-express ?                                  |
|--------------------------|----------------------------------------------------------|
| Recherche simple         | Barre du haut → option Simple                            |
| Recherche JSON (avancée) | Barre du haut → option Advanced                          |
| Trier                    | Mode Advanced → champ Sort                               |
| Projeter                 | Mode Advanced → champ Projection                         |
| Paginer                  | Mode Advanced → champs Skip et Limit                     |
| Compter                  | Pas d'option directe — utiliser un pipeline avec \$count |

### A.4. Index

| Action             | Où dans mongo-express ?                 |
|--------------------|-----------------------------------------|
| Lister les index   | Vue de la collection → onglet Indexes   |
| Créer un index     | Onglet Indexes → bouton Create Index    |
| Supprimer un index | Onglet Indexes → bouton Delete à droite |
| Statistiques       | Vue de la collection → onglet Stats     |

## Annexe B — Mémo des opérateurs MQL

### Comparaison

\$eq \$ne \$gt \$gte \$lt \$lte \$in \$nin

## Logique

\$and \$or \$not \$nor

## Existence et type

\$exists \$type

## Tableaux

\$all \$elemMatch \$size

## Regex

```
{ "email": { "$regex": "example.com$", "$options": "i" } }  
// $options: "i" = case-insensitive
```

## Étapes d'agrégation

\$match \$group \$sort \$limit \$skip \$project \$lookup \$unwind \$count

## Accumulateurs (\$group)

\$sum \$avg \$min \$max \$first \$last \$push \$addToSet

## Annexe C — Troubleshooting courant

### C.1. Docker / mongo-express

| Problème                                         | Solution                                                                |
|--------------------------------------------------|-------------------------------------------------------------------------|
| port 27017 already in use                        | Autre MongoDB tourne. Arrêter ou changer le port (27018:27017)          |
| port 8081 already in use                         | Changer le port mongo-express (8082:8081)                               |
| mongo-express bloqué sur Waiting for mongo:27017 | Attendre 30s, MongoDB met du temps au 1er lancement. Vérifier les logs. |
| Authentification refusée à l'accueil 8081        | Identifiants : student / nosql2025 (vérifier ME_CONFIG_BASICAUTH_*)     |
| Docker Desktop ne démarre pas sur Windows        | Vérifier WSL2 activé. Désinstaller / réinstaller Docker Desktop.        |
| Conteneur cassé, je veux tout repartir à zéro    | docker compose down -v puis docker compose up -d                        |

### C.2. Requêtes MongoDB

| Problème | Solution |
|----------|----------|
|----------|----------|

|                                   |                                                                                              |
|-----------------------------------|----------------------------------------------------------------------------------------------|
| Invalid JSON                      | Virgule en trop, guillemets manquants. Vérifier dans un validateur JSON en ligne.            |
| Zéro résultat                     | Vérifier que les données contiennent bien le cas cherché. La requête est peut-être correcte. |
| \$text ne fonctionne pas          | Il faut d'abord créer un <b>index text</b> sur le champ. Onglet Indexes.                     |
| Pipeline \$lookup ne renvoie rien | Vérifier que localField et foreignField sont bons et que les valeurs correspondent.          |
| Date au mauvais format            | Utiliser la notation {"\$date": "2025-09-15T10:30:00Z"} (format ISO 8601).                   |
| ObjectId au mauvais format        | Utiliser la notation {"\$oid": "65f3a8c2b4d8e1f23a4b5c6d"} (24 chars hex).                   |

## Annexe D — Pour aller plus loin

### D.1. Passer à mongosh (ligne de commande)

Si vous voulez utiliser le shell mongo officiel après ce TP, c'est facile :

```
# Se connecter au conteneur MongoDB en mode shell
docker exec -it cityflow-mongo mongosh -u admin -p cityflow2025

# Une fois connecté :
use cityflow
show dbs // lister les bases
show collections // lister les collections
db.dropDatabase() // supprimer la base courante

db.users.find({ age: { $gte: 18 } })
db.users.aggregate([
  { $group: { _id: "$city", count: { $sum: 1 } } }
])
// CREATE
db.coll.insertOne({...})
db.coll.insertMany([ {...}, {...} ])
// READ
db.coll.find() // tous
db.coll.find({ field: value })
db.coll.findOne({...})
db.coll.find({...}).limit(10).skip(20).sort({field: -1}) db.coll.countDocuments({...})
db.coll.distinct("field")
// UPDATE
db.coll.updateOne({filter}, {$set: {field: value}}) db.coll.updateMany({filter}, {$inc: {counter: 1}})
db.coll.replaceOne({filter}, {newDoc})
// DELETE
db.coll.deleteOne({filter})
db.coll.deleteMany({filter})
db.coll.drop() // supprimer la collection
```

```

db.coll.createIndex({field: 1}) // ascendant
db.coll.createIndex({field: -1}) // descendant
db.coll.createIndex({a: 1, b: -1}) // composé
db.coll.createIndex({field: 1}, {unique: true}) // unique
db.coll.createIndex({field: "text"}) // full-text
db.coll.createIndex({location: "2dsphere"}) // géo
db.coll.createIndex({ts: 1}, {expireAfterSeconds: 3600}) // TTL

db.coll.getIndexes() // lister
db.coll.dropIndex("nom_de_l_index") // supprimer

db.coll.find({...}).explain("executionStats") // plan d'exécution
db.coll.stats() // statistiques collection
db.serverStatus() // état serveur
db.currentOp() // opérations en cours
db.coll.totalIndexSize() // taille des index

```

## D.2. Schema Validation

Pour imposer un schéma minimal à une collection (recommandé en production), MongoDB supporte JSON Schema. Exemple à tester en bonus :

```

db.createCollection("users_v2", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["email", "firstName"],
      properties: {
        email: { bsonType: "string", pattern: "^.+@.+$" },
        firstName: { bsonType: "string", minLength: 2 },
        age: { bsonType: "int", minimum: 0, maximum: 150 }
      }
    }
  }
})

```

## D.3. Connecter MongoDB à du code

Une fois ce TP terminé, vous pourrez connecter MongoDB à du code applicatif. Les drivers officiels existent pour tous les langages. Exemples de strings de connexion :

```

// Node.js (mongoose ou driver natif)
const uri = "mongodb://admin:cityflow2025@localhost:27017";

# Python (pymongo)
from pymongo import MongoClient
client = MongoClient("mongodb://admin:cityflow2025@localhost:27017")

// .NET (MongoDB.Driver)
var client = new MongoClient("mongodb://admin:cityflow2025@localhost:27017");

```

## D.4. Ressources recommandées

- **MongoDB University** : [learn.mongodb.com](https://learn.mongodb.com) — certifications gratuites M001 et M201
- **Documentation officielle** : [mongodb.com/docs](https://mongodb.com/docs)
- **Aggregation Pipeline Builder** : test interactif via MongoDB Compass
- **Designing Data-Intensive Applications** (Kleppmann) — pour aller au-delà

### À retenir absolument de cette séance

- **Docker + mongo-express** = environnement MongoDB sans installation, idéal pour apprendre.
- **Document = entité métier complète**, pas une ligne de table normalisée.
- **Embedding vs Référencement** = LA décision clé. Embed les données toujours lues ensemble, référez les entités à existence propre.
- **Modélisez selon vos requêtes**, pas selon vos données. C'est l'inverse du réflexe SQL.
- **MQL** : opérateurs \$gt, \$in, \$elemMatch, notation pointée — la doc officielle est votre amie.
- **Pipeline d'agrégation** : \$match → \$group → \$sort → \$project. Comme des pipes Unix.
- **Indexez intelligemment** : champs filtrés/triés fréquents. Pas tout, pas rien.
- **Dénormalisation volontaire** : copier un nom ou un prix pour figer l'état au moment d'une transaction.